

M02 / PROTECTED ANSWERS

M02 Module Revision Test Answers

Standalone Zero-to-Job-Ready Data Engineer 2026

Study, practise, check and explain.

A complete learning route

Start with the lesson video. Read the worked explanation beside it, pause to reproduce the example, then solve the supplied task. The books contain your study notes; keep your own work and results as evidence.

Use the protected answers after saving your first attempt. A changed retry shows what you can now do independently. There is no forced timer during learning.

One course. M00-M19. 264 lessons and 50 project phases.

Contents

Tap a topic or use the PDF bookmarks. These numbers are the actual pages in this edition.

Read this first	4
How to use the protected answers	4
A01 - DE02-01 Database, table, row, column and query mental model	4
Expected answer / validation	4
A02 - DE02-02 SELECT, aliases and LIMIT	4
Expected answer / validation	4
A03 - DE02-03 WHERE and comparison operators	5
Expected answer / validation	5
A04 - DE02-04 AND, OR, NOT and parentheses	5
Expected answer / validation	5
A05 - DE02-05 ORDER BY, DISTINCT and result inspection	5
Expected answer / validation	5
A06 - DE02-06 NULL, IS NULL and COALESCE	6
Expected answer / validation	6
A07 - DE02-07 Arithmetic expressions and safe calculations	6
Expected answer / validation	6
A08 - DE02-08 Aggregate functions	6
Expected answer / validation	6
A09 - DE02-09 GROUP BY and grouped grain	7
Expected answer / validation	7
A10 - DE02-10 HAVING after aggregation	7

Expected answer / validation 7

A11 - DE02-11 | CASE for business categories 7

Expected answer / validation 7

A12 - DE02-12 | String and date basics 8

Expected answer / validation 8

A13 - DE02-13 | Simple subqueries and IN/EXISTS awareness 8

Expected answer / validation 8

A14 - DE02-14 | Debugging and validation workflow 8

Expected answer / validation 8

Read this first

Open this book after saving your own attempt. Compare the reasoning and result, repair the cause of a mistake, then close the answer and try changed inputs.

PROTECTED TEST ANSWERS

Use this file to validate and repair, not to erase your first attempt. Compare the expected control with what you actually observed, close this guide, then solve a changed retry if the competency was weak.

Coverage: 14 canonical lessons

Route: learner book -> practice -> revision -> closed-book test -> protected answers -> changed retry.

How to use the protected answers

Open only the lesson you attempted. The guide gives the expected control/behavior, not permission to copy. If you used it, close it and perform a changed retry before marking the lesson mastered.

A01 - DE02-01 | Database, table, row, column and query mental model

Expected answer / validation

```
SELECT order_id, quantity, unit_price
FROM course.retry_sales
ORDER BY order_id;
```

Rows are R201/3/25.00, R202/2/10.00, R203/3/25.00 and R204/2/15.00. IDs distinguish records; quantities measure units and support arithmetic. Four rows and three selected columns should appear. The correct input table matters as much as the selected columns.

After repair: close this guide and repeat the competency on a changed input/scenario. Preserve the original failed/partial attempt.

A02 - DE02-02 | SELECT, aliases and LIMIT

Expected answer / validation

```
SELECT order_id AS reference, item
FROM course.retry_sales
ORDER BY order_id DESC
LIMIT 2;
```

R204 Ruler then R203 Folder should appear. A new query against course.retry_sales still names order_id. The alias did not rename the stored field. Descending sort changes what "first two" means for the limited output.

After repair: close this guide and repeat the competency on a changed input/scenario. Preserve the original failed/partial attempt.

A03 - DE02-03 | WHERE and comparison operators

Expected answer / validation

```
SELECT order_id, ordered_on
FROM course.retry_sales
WHERE ordered_on < DATE '2026-11-02'
ORDER BY order_id;
```

R201 and R202, both November 1, remain. "Before" excludes the boundary date. R203 on November 2 would appear with <=, which would answer a different question.

After repair: close this guide and repeat the competency on a changed input/scenario. Preserve the original failed/partial attempt.

A04 - DE02-04 | AND, OR, NOT and parentheses

Expected answer / validation

```
SELECT order_id, category, quantity
FROM course.retry_sales
WHERE NOT (category = 'Paper')
      AND quantity >= 2
ORDER BY order_id;
```

R202 Writing/2 and R204 Tools/2 qualify. Both R201 and R203 are Paper and fail the first test. Parentheses around the NOT operand make the intended scope visible.

After repair: close this guide and repeat the competency on a changed input/scenario. Preserve the original failed/partial attempt.

A05 - DE02-05 | ORDER BY, DISTINCT and result inspection

Expected answer / validation

```
SELECT DISTINCT item, unit_price
FROM course.retry_sales
ORDER BY item, unit_price;
```

```
SELECT order_id, quantity
FROM course.retry_sales
ORDER BY quantity DESC, order_id
LIMIT 2;
```

The four item/price combinations are Folder/25, Notebook/25, Pen/10 and Ruler/15. Folder and Notebook remain separate because item differs even though price matches. The preview is R201/3 then R203/3, ordered by their IDs within the quantity tie.

After repair: close this guide and repeat the competency on a changed input/scenario. Preserve the original failed/partial attempt.

A06 - DE02-06 | NULL, IS NULL and COALESCE

Expected answer / validation

```
SELECT order_id
FROM course.retry_sales
WHERE customer_id IS NULL
ORDER BY order_id;
```

```
SELECT order_id
FROM course.retry_sales
WHERE discount_pct IS NULL
ORDER BY order_id;
```

R204 lacks a customer ID; R203 lacks a discount. Missingness belongs to individual fields. A row may contain known quantities, unknown customer identity and a known zero discount at the same time. Missing one field does not make every other value unusable.

After repair: close this guide and repeat the competency on a changed input/scenario. Preserve the original failed/partial attempt.

A07 - DE02-07 | Arithmetic expressions and safe calculations

Expected answer / validation

```
SELECT order_id, quantity * unit_price AS gross_amount
FROM course.retry_sales
ORDER BY order_id;
```

```
SELECT 3.0 / NULLIF(0, 0) AS return_rate;
```

Gross amounts are 75, 20, 75 and 30. The return_rate is NULL because the denominator is zero. That is an undefined ratio, not evidence that no returns occurred. The numerator explicitly says three units were returned.

After repair: close this guide and repeat the competency on a changed input/scenario. Preserve the original failed/partial attempt.

A08 - DE02-08 | Aggregate functions

Expected answer / validation

```
SELECT COUNT(*) AS sale_rows, COUNT(is_paid) AS known_payment_rows,
       SUM(quantity) AS units, SUM(quantity * unit_price) AS gross_total,
       AVG(quantity * unit_price) AS mean_sale
FROM course.retry_sales;
```

The values are 4, 3, 10, 200 and 50. Mean sale amount divides 200 by four sales. Dividing by ten units would answer the different question of revenue per unit: 20. Names and units prevent these measures being mistaken for each other.

After repair: close this guide and repeat the competency on a changed input/scenario. Preserve the original failed/partial attempt.

A09 - DE02-09 | GROUP BY and grouped grain

Expected answer / validation

```
SELECT ordered_on, COUNT(*) AS sale_rows, SUM(quantity) AS units,  
       SUM(quantity * unit_price) AS gross_total  
FROM course.retry_sales  
GROUP BY ordered_on  
ORDER BY ordered_on;
```

November 1 is 2 rows/5 units/95; November 2 is 1/3/75; November 3 is 1/2/30. Across the three day summaries there are four source rows, ten units and 200 gross amount. Source record count, quantity and currency have different units and should remain separately named.

After repair: close this guide and repeat the competency on a changed input/scenario. Preserve the original failed/partial attempt.

A10 - DE02-10 | HAVING after aggregation

Expected answer / validation

```
SELECT category, COUNT(*) AS sale_rows,  
       SUM(quantity * unit_price) AS gross_total  
FROM course.retry_sales  
GROUP BY category  
HAVING COUNT(*) >= 2  
ORDER BY category;
```

Only Paper remains, with two rows and gross 150. Writing and Tools each have one row. The number of records in a category is a property of the group; it is not an original field on each sale.

After repair: close this guide and repeat the competency on a changed input/scenario. Preserve the original failed/partial attempt.

A11 - DE02-11 | CASE for business categories

Expected answer / validation

```
SELECT order_id,  
       CASE WHEN quantity * unit_price >= 75 THEN 'High' WHEN quantity * unit_price >= 30 THEN 'Mid' ELSE 'Low'  
       END AS amount_group  
FROM course.retry_sales  
ORDER BY order_id;
```

R201 and R203 are High at exactly 75, R204 Mid at exactly 30, and R202 Low at 20. These boundary

cases are more useful than checking only a value far from every threshold. First matching branch wins.

After repair: close this guide and repeat the competency on a changed input/scenario. Preserve the original failed/partial attempt.

A12 - DE02-12 | String and date basics

Expected answer / validation

```
SELECT item, UPPER(item) AS item_upper,
       EXTRACT(MONTH
              FROM ordered_on) AS month_number, ordered_on + 7 AS follow_up_date
FROM course.retry_sales
WHERE order_id = 'R203';
```

Expected Folder, FOLDER, 11 and 2026-11-09. UPPER transforms the result expression only; it does not rewrite the source item. The stored date is November 2 and adding seven days produces November 9.

After repair: close this guide and repeat the competency on a changed input/scenario. Preserve the original failed/partial attempt.

A13 - DE02-13 | Simple subqueries and IN/EXISTS awareness

Expected answer / validation

```
SELECT order_id, quantity * unit_price AS gross_amount
FROM course.retry_sales
WHERE quantity * unit_price >=
      (SELECT AVG(quantity * unit_price)
       FROM course.retry_sales)
ORDER BY order_id;
```

The threshold is 50, so R201/75 and R203/75 qualify. EXISTS answers true/false about the presence of rows; it does not supply the numeric mean. The subquery's output shape must fit the outer question.

After repair: close this guide and repeat the competency on a changed input/scenario. Preserve the original failed/partial attempt.

A14 - DE02-14 | Debugging and validation workflow

Expected answer / validation

```
SELECT CASE WHEN is_paid = TRUE THEN 'Paid' WHEN is_paid = FALSE THEN 'Unpaid' ELSE 'Unknown'
       END AS payment_group, COUNT(*) AS sale_rows,
       SUM(quantity * unit_price) AS gross_total
FROM course.retry_sales
GROUP BY CASE WHEN is_paid = TRUE THEN 'Paid' WHEN is_paid = FALSE THEN 'Unpaid' ELSE 'Unknown'
       END
ORDER BY payment_group;
```

Paid is 2/150, Unknown 1/30 and Unpaid 1/20. Counts total four; amounts total 200. Labelling the unknown row as unpaid would show 50 unpaid, mixing one known unpaid sale with one whose status has not been recorded. A reconciled grand total would still be 200, so total agreement alone would miss that meaning error.

After repair: close this guide and repeat the competency on a changed input/scenario. Preserve the original failed/partial attempt.